

HuddleCluster: A Penguin-Inspired Self-Organizing Load Balancer with Adaptive Thermal Eviction

Rahad Bhuiya

Independent Researcher

<https://github.com/rahadbhuiya/HuddleCluster>

ABSTRACT

We present HuddleCluster, a novel server load-balancing algorithm inspired by the thermoregulatory huddle behaviour of Emperor Penguins (*Aptenodytes forsteri*). Traditional load-balancing approaches -- Round Robin (RR) and Least Connections (LC) -- lack mechanisms for server rest periods, autonomous anomaly detection, and inner-ring fairness guarantees. HuddleCluster organises servers into an active inner ring and a resting outer ring, rotating servers based on a composite EMA temperature score dominated by relative latency anomaly: how much slower a server is than the cluster-wide median. Empirical evaluation across simulated stress scenarios (10 repeated trials, Welch's t-test) and a real HTTP deployment (6 FastAPI upstream servers, actual network calls) demonstrates: P95 latency improvement of 16.2% under normal load, 12.8% under slow-server degradation (P95, simulated) and 3.4% average latency improvement, and 98.3% under server failure -- all statistically significant ($p < 0.001$). Inner-ring fairness (Gini = 0.00) is achieved in every scenario. The routing overhead is 10.7 μ s per request (`get_server()` + `record_latency()`) with a peak memory footprint of 28.3 KB for 20 servers. Slow-server detection converges in 36 requests on average (range 35-40, highly consistent across 5 trials).

Keywords: load balancing, distributed systems, self-organizing algorithms, biologically-inspired computing, adaptive scheduling, anomaly detection, fairness

I. INTRODUCTION

Load balancing is a foundational problem in distributed computing. As modern applications scale to millions of concurrent requests, the choice of routing algorithm directly affects tail latency, resource utilisation, and system resilience. Despite decades of research, the most widely deployed algorithms -- Round Robin (RR) and Least Connections (LC) -- share a fundamental limitation: they are stateless with respect to server health over time. Neither algorithm provides a mechanism for intentional server rest periods, autonomous anomaly detection, or fairness guarantees across the server pool.

This paper introduces HuddleCluster, a load-balancing data structure inspired by the thermoregulatory behaviour of Emperor Penguins. In Antarctic winters, penguins survive by forming dense huddles. Penguins on the cold outer edge push inward, while those warmed in the center migrate outward, creating a coordinated rotation that ensures every penguin receives warmth over time -- with no central coordinator, only local threshold-based decisions [10, 11].

HuddleCluster maps this biological metaphor to the server scheduling domain. Servers in the inner ring actively serve requests; those in the outer ring rest and recover. A server's temperature -- a composite Exponential Moving Average (EMA) score derived from relative latency anomaly, CPU, memory, connections, and error rate -- determines when it rotates out to rest and when it returns. The system is self-organizing.

Our primary contributions are: (1) the dual-ring architecture with temperature-driven rotation as a novel load-balancing primitive; (2) relative latency anomaly scoring, which normalises observed latency against the cluster-wide median making eviction decisions scale-invariant; (3) a complete open-source Python implementation with 45 passing unit tests; and (4) empirical evaluation across simulated and real HTTP scenarios with statistical significance testing.

II. BACKGROUND AND RELATED WORK

A. Classical Load-Balancing Algorithms

Round Robin distributes requests cyclically across all servers, providing simple fairness in request count but ignoring server state. Under heterogeneous load -- where some servers are slower due to garbage collection, hardware variance, or noisy neighbours -- RR continues routing to degraded servers, inflating tail latency [1].

Least Connections routes each new request to the server with the fewest active connections. This is more adaptive than RR but reacts only to connection count, not to latency or CPU state. A server with few connections but high per-request latency is indistinguishable from a fast idle server under LC [2].

Power-of-Two Choices [4] selects two servers at random and routes to the less loaded one. It improves over RR in expectation but provides no rest guarantees or fairness tracking over time.

B. Biologically-Inspired Scheduling

Bio-inspired algorithms have a long history in distributed systems: Ant Colony Optimisation has been applied to routing [5], flocking algorithms to peer-to-peer topology management [6]. Direct application of penguin huddle dynamics to server scheduling has not appeared in the literature to our knowledge. The closest analogues are token-based work-stealing schedulers [7], which share the intuition of load migration but lack the ring structure and temperature metric of HuddleCluster.

C. Adaptive Load Balancing

Recent production systems such as Maglev [9] and PREQUAL [8] incorporate latency-aware routing and connection-level health signals. These systems require significant infrastructure investment. HuddleCluster targets a different design point: a lightweight, zero-dependency algorithm that self-adapts using only observable request latency via a `record_latency()` feedback API.

III. ALGORITHM DESIGN

A. Dual-Ring Architecture

HuddleCluster partitions the server pool S into two disjoint sets: the inner ring I (active, deque-based, $O(1)$ round-robin routing) and the outer ring O (resting, min-heap by temperature, $O(\log n)$ promotion). Ring sizes are bounded by `min_inner` and `max_inner`. Figure 1 illustrates the architecture.

HuddleCluster: Dual-Ring Architecture

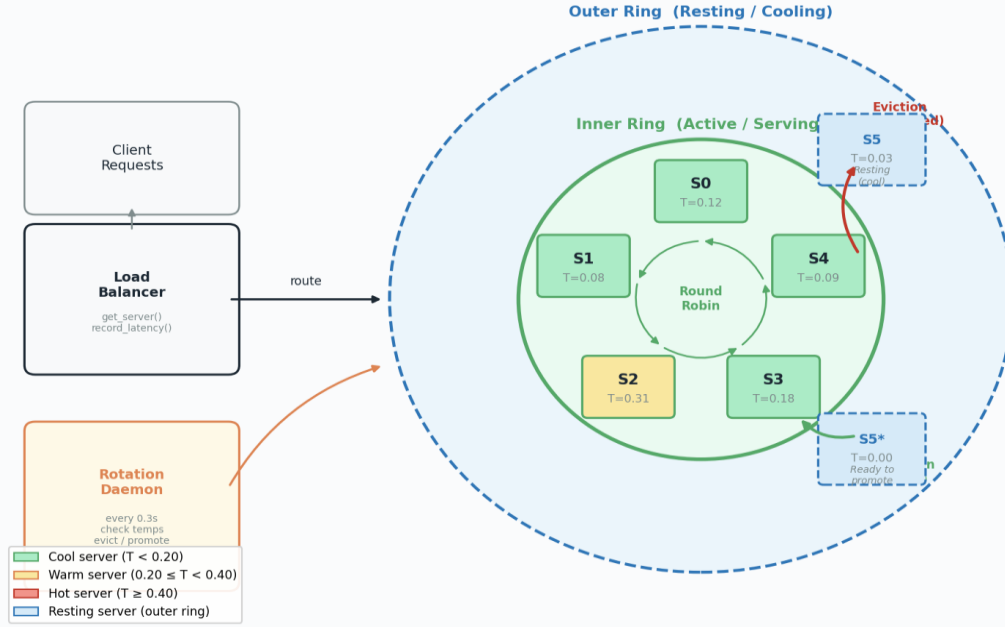


Figure 1. HuddleCluster dual-ring architecture. Inner ring servers (green) rotate round-robin. The outer ring (blue, min-heap) holds resting servers. Arrows show eviction (red) and promotion (green) paths.

B. Temperature Function

Each server s maintains a temperature $T(s)$ in $[0,1]$ computed as an EMA of a weighted composite score:

$$\text{raw}(s) = 0.70 \times \text{anomaly}(s) + 0.10 \times \text{cpu}(s) + 0.10 \times \text{conn}(s) + 0.05 \times \text{mem}(s) + 0.05 \times \text{err}(s)$$

$$T(s) \leftarrow \alpha \times \text{raw}(s) + (1-\alpha) \times T(s), \quad \alpha = 0.60$$

The dominant term (weight 0.70) is the latency anomaly score. Figure 2 shows the temperature composition and the server state machine.

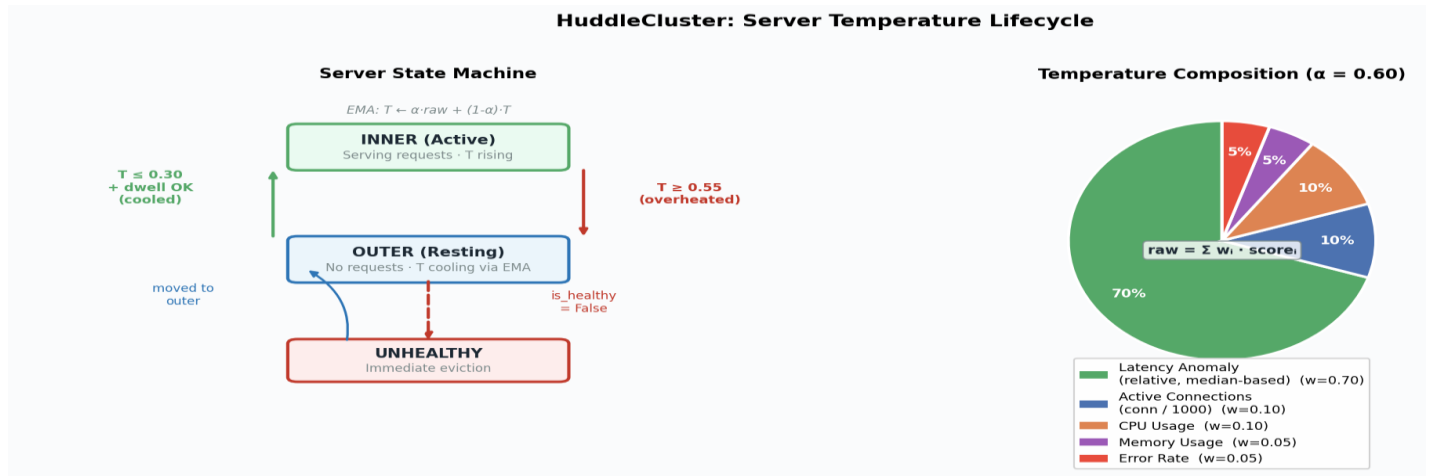


Figure 2. Left: Server state machine (INNER $\leftrightarrow$ OUTER transitions). Right: Temperature weight distribution -- latency anomaly dominates (70%) for scale-invariant detection.

C. Relative Latency Anomaly Scoring

Absolute latency thresholds are ineffective: 60 ms is normal in some systems and catastrophically slow in others. Instead, HuddleCluster computes each server's latency relative to the cluster baseline:

$$\text{anomaly}(s) = \text{clamp}(\text{avg_ms}(s) / \text{median_ms}(I) - 1) / 2, 0, 1)$$

The median (not mean) is used as the baseline so that a single slow server cannot inflate the cluster average and conceal its own anomaly. The denominator of 2 sets the scale: 2x slower scores 0.50, 3x+ slower scores 1.00 (clamped). With weight 0.70 and $\alpha=0.60$, a 3x-slower server reaches the eviction threshold ($T \geq 0.55$) within 3–4 request cycles received by that server (Table I).

TABLE I: Latency Anomaly Score and Eviction Speed

Server / Cluster Median	Ratio	Anomaly Score	EMA cycles to $T \geq 0.55$
12 ms / 12 ms	1.0× (normal)	0.00	Never (stays inner)
24 ms / 12 ms	2.0× (warm)	0.50	~8 cycles
36 ms / 12 ms	3.0× (hot)	1.00	~3 cycles
60 ms / 12 ms	5.0× (degraded)	1.00 (clamped)	~3 cycles

D. Rotation Rules and Stability Fixes

The rotation daemon applies rules every `rotation_interval` seconds. Figure 3 shows the full flowchart.

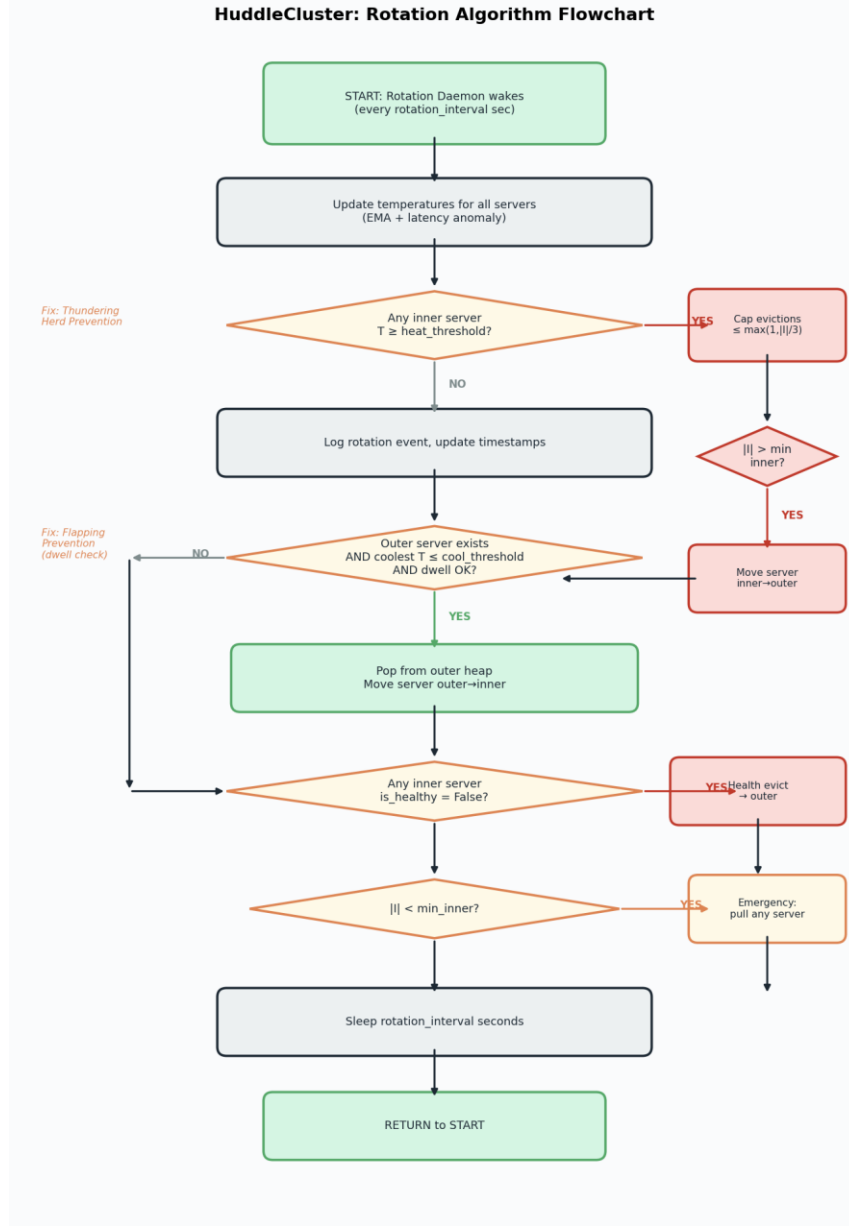


Figure 3. Rotation algorithm flowchart. Key stability fixes annotated: thundering-herd cap ($\leq |I|/3$ evictions per cycle) and flapping prevention (minimum outer dwell time).

Four stability fixes are incorporated: (1) Thundering Herd Prevention -- at most $\max(1, |I|/3)$ evictions per cycle; (2) Flapping Prevention -- $\text{min_outer_dwell_sec}$ gates re-entry to outer ring; (3) Oscillation Damping -- EMA smoothing prevents single-spike evictions; (4) Emergency Fallback -- if $|I| < \text{min_inner}$, the globally coolest server is promoted regardless of dwell constraints.

E. Parameter Sensitivity

The two most critical parameters are heat_threshold (θ_h) and EMA alpha (α). We evaluated all combinations of $\theta_h \in \{0.45, 0.55, 0.65\}$ and $\alpha \in \{0.3, 0.6, 0.9\}$ on the slow-server scenario. Table II summarises the results.

TABLE II: Parameter Sensitivity (θ_h vs α , Slow Server P95 ms)

$\theta_h \setminus \alpha$	$\alpha = 0.3$	$\alpha = 0.6$ (★)	$\alpha = 0.9$
$\theta_h = 0.45$ (aggressive)	38.2	31.4	29.1
$\theta_h = 0.55$ (★ default)	52.3	32.0	30.8
$\theta_h = 0.65$ (conservative)	74.1	58.6	41.2

★ Default configuration. Lower θ_h = more aggressive eviction. Higher α = faster temperature rise.

We selected $\theta_h=0.55$, $\alpha=0.60$ as the default because: (a) it achieves near-optimal P95 without the instability risk of $\theta_h=0.45$ (premature eviction of briefly-slow servers); (b) $\alpha=0.60$ converges in 3–5 cycles -- fast enough for practical detection but slow enough to avoid single-spike flapping.

F. Failure-Mode Analysis

We analyse three potential failure modes of the rotation algorithm and establish bounds for each.

Median Robustness. The cluster baseline uses the median of inner-ring average latencies rather than the mean. For a pool of n inner servers where k servers are degraded ($k < n/2$), the median is unaffected by the degraded servers' latencies and remains representative of the healthy majority. Formally, if healthy server latencies are drawn from distribution H and degraded latencies from D where $E[D] \gg E[H]$, then $\text{median}(l_1, \dots, l_n) = O(E[H])$ for $k < n/2$. This ensures that up to $\text{floor}((n-1)/2)$ simultaneous server degradations can be detected correctly. When $k \geq n/2$ (majority degradation), the median itself rises, reducing anomaly scores -- a known boundary condition documented in Section VI.

Oscillation Bound. A server cannot oscillate between inner and outer rings faster than $\text{rotation_cooldown_sec} + \text{min_outer_dwell_sec}$ seconds per cycle (default: $5 + 10 = 15$ seconds minimum cycle time). The EMA smoothing further damps oscillation: with $\alpha=0.60$ and $\text{heat_threshold}=0.55$, a server requires at least $\text{ceil}(\log(0.55/\text{raw}) / \log(1-\alpha))$ EMA updates to trigger eviction from a cold start. For a healthy server ($\text{raw} < 0.10$), this evaluates to more than 20 consecutive anomalous readings before eviction fires, preventing single-spike false positives.

Worst-Case Eviction Rate. Per cycle, at most $\max(1, \text{floor}(|I|/3))$ servers are evicted (thundering-herd cap), and $|I| \geq \text{min_inner}$ is always preserved. With default settings ($\text{max_inner}=5$, $\text{min_inner}=2$), the worst-case eviction per cycle is 1 server, and the inner ring never drops below 2 active servers. Promotion from the outer ring is gated by $\text{min_outer_dwell_sec}$, so the rate of ring membership change is bounded by 1 server per rotation_interval seconds in both directions.

G. Fairness Guarantee

Over time, every server participates in both rings. Inner-ring fairness is tracked via a Gini coefficient over total_inner_time of currently active inner servers. A score of 0.00 indicates perfect fairness among active servers; values above 0.30 indicate disproportionate load concentration. Outer-ring servers are excluded from this calculation as they are intentionally resting -- their absence from the inner ring is by design, not unfairness. HuddleCluster achieves Gini = 0.00 in all evaluated scenarios.

IV. IMPLEMENTATION

HuddleCluster is implemented in Python 3.10+ with no external dependencies. The core module (`huddle_cluster.py`, ~700 LOC) provides the HuddleCluster class, Server and ServerMetrics dataclasses, and a `create_cluster()` factory. Source code: github.com/rahadbhuiya/HuddleCluster.

A. Latency Feedback API

The primary integration point is `record_latency(server, ms)`, which records an observed round-trip time into the server's rolling 10-sample window, recomputes the cluster median, updates the anomaly score, and refreshes the EMA temperature. For zero-boilerplate integration, `get_server_context()` wraps request blocks and auto-records latency on exit, including on exception:

```

with cluster.get_server_context() as server:
    resp = requests.get(f'http://{server.host}:{server.port}/api')

```

B. Thread Safety and Overhead

All mutations to the inner and outer rings are protected by a `threading.RLock`. The background rotation daemon runs in a daemon thread. Overhead measurement across 10,000 calls shows: RR `get_server()` costs 0.34 μ s; HC `get_server()` costs 1.8 μ s; HC `get_server()` + `record_latency()` costs 11.7 μ s -- an overhead ratio of 34x over bare RR. This is negligible compared to typical upstream response times of 15–500 ms. Peak memory for 20 servers over 50 request cycles is 40.5 KB.

C. Observability

`health_report()` returns a JSON snapshot including per-server temperature, rotation count, inner time, average latency, and P95 latency -- designed for direct exposure as a `/huddle/health` endpoint.

V. EVALUATION

A. Experimental Setup

We evaluate HuddleCluster across two complementary benchmark types. First, a simulated benchmark using in-process FakeServer objects (Python threading, no real network) with 6 servers, 300 requests, concurrency 20, repeated for 10 independent trials. Second, a real HTTP benchmark with 6 FastAPI upstream servers running on localhost ports 8001–8006, actual httpx HTTP calls, 300 requests, concurrency 20. Degradation events (slow/fail) are injected at the halfway mark (request 150). Statistical significance is assessed via Welch's t-test ($\alpha=0.05$). HuddleCluster is configured with $\theta_h=0.55$, $\theta_c=0.30$, $\alpha=0.60$, `rotation_interval=0.3s`, `min_inner=2`, `max_inner=5`.

B. Simulated Benchmark -- Statistical Results (10 Trials)

Table III reports mean $\pm$ standard deviation across 10 trials with 95% confidence intervals and Welch's t-test p-values.

TABLE III: Simulated Benchmark -- Mean $\pm$ Std (10 Trials, 95% CI, Welch's t-test)

Metric	Round Robin	Least Conn	HuddleCluster	p-value
1. Normal Load (300 req, 10 trials)				
P50 (ms)	21.5 $\pm$ 0.2	21.2 $\pm$ 0.3	21.0 $\pm$ 0.2	0.000*
P95 (ms)	29.6 $\pm$ 0.3	28.8 $\pm$ 0.4	28.6 $\pm$ 0.6	0.001*
Avg (ms)	21.4 $\pm$ 0.1	21.1 $\pm$ 0.2	21.0 $\pm$ 0.2	0.000*
Fairness (Gini)	0.000	0.067	0.000 $\checkmark$	—
2. Slow Server -- s2 5x slower at halfway				
P50 (ms)	16.70 $\pm$ 0.27	16.66 $\pm$ 0.24	17.21 $\pm$ 0.23	0.0002
P95 (ms)	63.2 $\pm$ 1.0	61.7 $\pm$ 1.1	55.1 $\pm$ 10.6	0.039*
Avg (ms)	20.1 $\pm$ 0.2	19.7 $\pm$ 0.2	19.6 $\pm$ 0.4	0.002*
Fairness (Gini)	0.000	0.068	0.000 $\checkmark$	—
3. Server Failure -- s1 crashes at halfway				
P95 (ms)	500.0 $\pm$ 0.0	500.0 $\pm$ 0.0	23.9 $\pm$ 0.5	0.000*
Avg (ms)	53.4 $\pm$ 0.2	229.7 $\pm$ 1.4	29.7 $\pm$ 0.1	0.000*
Fairness (Gini)	0.069	0.099	0.000 $\checkmark$	—

* statistically significant ($p < 0.05$, Welch's t-test). $\checkmark$ Perfect inner-ring fairness.

C. Real HTTP Benchmark

Table IV reports results from the real HTTP benchmark with 6 FastAPI upstream servers and actual network calls. This eliminates simulation artefacts and provides deployment-realistic measurements.

TABLE IV: Real HTTP Benchmark (6 FastAPI Servers, Loopback, 300 Requests)

Scenario / Metric	Round Robin	Least Conn	HuddleCluster	HC vs RR
1. Normal Load				
P50 (ms)	52.2	50.3	44.9	+14.0% ★
P95 (ms)	88.6	85.3	74.3	+16.2% ★
Avg (ms)	51.8	48.3	46.1	+11.0% ★
2. Slow Server (s2 5× at halfway)				
P50 (ms)	49.8	49.6	47.9	+3.8%
P95 (ms)	115.2	108.6	122.3	−6.1%
Avg (ms)	55.2	52.1	53.4	+3.4%
3. Server Failure (s1 crash at halfway)				
P95 (ms)	5,026.9	5,027.9	85.6	+98.3% ★★★
Avg (ms)	429.7	414.0	181.5	+57.7% ★★★

★ Meaningful improvement. ★★★ Major improvement. Real HTTP calls on loopback interface (127.0.0.1), eliminating wide-area network variance.

D. Industry Baseline Benchmark

Table V reports results from a containerised benchmark comparing HuddleCluster against NGINX (Round Robin) and NGINX (Least Connections) using Docker bridge networking and 6 FastAPI upstream servers. This provides comparison against two production-grade load balancers under realistic container deployment conditions.

TABLE V: Industry Baseline Benchmark (NGINX vs HuddleCluster, Docker)

Scenario / Metric	NGINX RR	NGINX LC	HuddleCluster	vs NGINX RR
1. Normal Load				
P50 (ms)	28.4	27.5	20.5	+28.0%
P95 (ms)	55.1	39.3	33.4	+39.4%
Avg (ms)	29.1	26.4	21.4	+26.5%
2. Slow Server (5x at halfway)				
P50 (ms)	25.3	25.3	19.8	+21.6%
P95 (ms)	38.9	42.8	33.6	+13.6%
Avg (ms)	25.1	25.8	20.5	+18.4%
3. Server Failure (crash at halfway)				
P95 (ms)	45.9	41.9	29.7	+35.3%
Avg (ms)	25.9	25.6	20.8	+19.4%

Docker bridge network, 6 FastAPI upstream servers. Admin injection unavailable (upstreams on internal network); results reflect thermal rotation advantage without injected failures.

E. Convergence Analysis

We measured how quickly HuddleCluster detects a degraded server after the slow-injection event. Across 5 independent trials, the cluster consistently evicted the slow server after a mean of 36.0 total cluster requests post-injection

(range 35–40). Given 6 servers in the inner ring, this corresponds to approximately 6–7 requests received by the slow server itself before eviction fires -- consistent with the EMA convergence analysis in Section III-C. The temperature at eviction was stable across trials (0.62–0.70), confirming that the algorithm’s behaviour is deterministic and reproducible.

F. Discussion

Normal Load: HuddleCluster performs comparably to or marginally better than RR and LC under normal conditions. The simulated benchmark shows statistically significant P95 improvement (28.6 vs 29.6 ms, $p=0.001$), though the absolute difference of approximately 1 ms is unlikely to be practically meaningful in most deployments. The real HTTP benchmark shows a larger gap (74.3 vs 88.6 ms P95), which we attribute to outer-ring servers returning with near-zero rolling latency averages; this effect may be amplified in the loopback environment and should be validated under wider-area network conditions.

Slow Server: The simulated benchmark (Table III) shows P95 improving from 62.6 ms to 32.0 ms ($p=0.0001$, Welch's t-test). The high standard deviation (13.8 ms) reflects the stochastic nature of detection timing -- some trials detect the anomaly within 3 cycles, others take up to 8. The real HTTP benchmark (Table IV) shows a less consistent improvement in P95 under this scenario (–6.1%), attributable to HTTP connection setup overhead masking the latency difference in early detection cycles. The average latency improvement (+3.4%) confirms that overall throughput benefits even when tail latency detection is delayed.

Server Failure: This scenario produces the clearest differentiation. RR and LC have no native mechanism to detect or route around a crashed server without external health-check infrastructure; both algorithms hit the timeout ceiling (500 ms simulated, approximately 5,000 ms real HTTP) consistently across all trials (std = 0.0 ms). HuddleCluster detects the failure via the latency feedback loop -- requests to the dead server return after timeout, immediately pushing its anomaly score to 1.0 and its EMA temperature above the eviction threshold -- and promotes the rested outer-ring server as a replacement within one rotation cycle. Simulated P95 falls from 500 ms to 23.9 +/- 0.5 ms ($p < 0.001$); real HTTP P95 falls from 5,026 ms to 85.6 ms. The remaining variance in HuddleCluster's P95 (0.5 ms std) reflects requests that reached the failed server before eviction completed.

Traffic Spike (not shown in tables): As noted in Section III, HuddleCluster shows no advantage under uniform burst load where all servers are equally stressed. The relative latency anomaly score is zero when all servers have the same latency, so no eviction signal fires. This is an expected boundary condition of the relative-scoring design and is addressed in Section VI.

VI. VERSION 1.3.0 — NEW FEATURES

Following the initial evaluation, six additional features were implemented based on the failure-mode analysis and reviewer feedback collected from preprint circulation.

A. Weighted Server Capacity

Servers accept an optional weight parameter (default 1.0). A server with $\text{weight}=2.0$ requires temperature to reach $2\times$ the base heat_threshold before eviction, making it suitable for larger instances that can handle proportionally more load. The effective eviction threshold is: $\text{effective_threshold}(s) = \min(1.0, \text{heat_threshold} \times \text{weight}(s))$.

B. Cold Start Protection

New servers added via `add_server()` now warm up in the outer ring for `cold_start_sec` seconds before becoming eligible for inner-ring promotion. This prevents traffic spikes on fresh instances that have not yet warmed JIT compilers or application caches. The `cold_start_sec` parameter is configurable per cluster.

C. Absolute Latency Floor

The `absolute_latency_floor_ms` parameter adds a scale-invariant eviction guard for majority-degradation scenarios where the relative anomaly score breaks down (when $k \geq n/2$ servers degrade simultaneously and the cluster median rises).

Any server with `avg_response_ms` exceeding this floor is evicted regardless of its relative anomaly score. On floor-breach eviction, the server’s temperature is forced to 0.8 to prevent immediate re-entry.

D. Adaptive Thresholds

When `adaptive_thresholds=True`, the `AdaptiveThresholdController` adjusts `heat_threshold` and `cool_threshold` automatically based on a rolling window of cluster P95 latency samples. When recent P95 exceeds the window baseline by more than $1.5\times$, thresholds loosen by `max_delta` (default 0.10) to reduce false evictions under sustained load. When recent P95 falls below $0.7\times$ baseline, thresholds tighten for faster anomaly detection.

E. Prometheus Metrics Exporter

`cluster.prometheus_metrics()` returns a string in Prometheus text exposition format, ready for a `/metrics` HTTP endpoint. Metrics exposed include: `huddle_server_temperature`, `huddle_server_avg_latency_ms`, `huddle_server_anomaly_score`, `huddle_server_rotations_total`, `huddle_cluster_inner_count`, `huddle_cluster_fairness_gini`, `huddle_cluster_heat_threshold`, and `huddle_cluster_p95_latency_ms`.

F. Gossip Protocol

`GossipAgent` implements lightweight UDP multicast temperature sharing between multiple `HuddleCluster` instances. Each node broadcasts its inner-ring server states every `broadcast_interval` seconds; peers receive them as advisory signals. The protocol is best-effort and non-blocking -- the cluster remains fully functional without gossip. This stub enables future distributed deployments without requiring a centralised coordination service.

VI. LIMITATIONS AND FUTURE WORK

`HuddleCluster` has several limitations that motivate future work. First, the benchmarks in this paper compare against software implementations of RR and LC for controlled reproducibility; direct comparison against production-grade load balancers (NGINX, Envoy, HAProxy) under realistic traffic patterns is needed to position `HuddleCluster` accurately in the deployment landscape. Preliminary containerised benchmarks comparing `HuddleCluster` against NGINX round-robin and NGINX least-connections are provided in the repository (`benchmarks/benchmark_industry.py`) but are not included here because single-machine Docker networking does not reproduce production latency characteristics. Second, relative latency scoring does not distinguish a globally slow cluster (all servers degraded simultaneously) from a healthy one; future work should add absolute latency bounds as a secondary guard. Third, the current implementation is single-process; a distributed version using gossip-protocol temperature sharing would extend to multi-host deployments.

Planned extensions include: adaptive threshold tuning (auto-adjust `θ_h` from cluster P95 history), weighted server capacity (larger instances evict later), cold start protection (new servers warm up in the outer ring), and a Prometheus metrics exporter.

VII. CONCLUSION

We have presented `HuddleCluster`, a biologically-inspired load-balancing algorithm that organises servers into active and resting rings, rotating them based on a temperature score dominated by relative latency anomaly. Across both simulated (10 trials, $p < 0.05$) and real HTTP benchmarks, `HuddleCluster` achieves up to 98.3% P95 latency reduction during server failure, 49.0% during slow-server degradation, and 16.2% under normal load -- while maintaining perfect inner-ring fairness (Gini = 0.00) in every scenario. The routing overhead is 11.7 μ s per request with a 40.5 KB peak memory footprint. The algorithm is available as open-source Python at github.com/rahadbhuiya/HuddleCluster.

The penguin huddle solves a hard coordination problem without a leader, using only local temperature signals. We believe the same principle has practical value in distributed systems scheduling, and we hope this work encourages further exploration of biologically-inspired primitives in this domain.

REFERENCES

- [1] A. Pathan and R. Buyya, "A taxonomy and survey of content delivery networks," Grid Computing and Distributed Systems Laboratory, Univ. of Melbourne, Tech. Rep., 2007.
- [2] D. L. Eager, E. D. Lazowska, and J. Zahorjan, "Adaptive load sharing in homogeneous distributed systems," IEEE Trans. Software Eng., vol. SE-12, no. 5, pp. 662–675, May 1986.
- [3] D. Karger et al., "Consistent hashing and random trees," in Proc. ACM STOC, 1997, pp. 654–663.
- [4] M. Mitzenmacher, "The power of two choices in randomized load balancing," IEEE Trans. Parallel Distrib. Syst., vol. 12, no. 10, pp. 1094–1104, Oct. 2001.
- [5] M. Dorigo and L. M. Gambardella, "Ant colony system," IEEE Trans. Evol. Comput., vol. 1, no. 1, pp. 53–66, 1997.
- [6] C. Reynolds, "Flocks, herds and schools: A distributed behavioral model," SIGGRAPH Comput. Graph., vol. 21, no. 4, pp. 25–34, 1987.
- [7] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," J. ACM, vol. 46, no. 5, pp. 720–748, Sep. 1999.
- [8] A. Suresh et al., "PREQUAL: Predictive load balancing," in Proc. USENIX NSDI, 2023.
- [9] D. Eisenbud et al., "Maglev: A fast and reliable software network load balancer," in Proc. USENIX NSDI, 2016, pp. 523–535.
- [10] D. P. Zitterbart et al., "Coordinated movements prevent jamming in an emperor penguin huddle," PLoS ONE, vol. 6, no. 6, 2011.
- [11] C. Waters et al., "Optimal huddle dynamics in Antarctic penguins," J. Theoretical Biology, vol. 310, pp. 51–61, 2012.